

SQL Server upgrade

📌 1. PRE-UPGRADE PREPARATION

1. Verify system requirements for target SQL Server version.

<https://learn.microsoft.com/en-us/sql/sql-server/> -- use this link to see the requirement for the server

commands to check what you have:

- wmic cpu get name, NumberOfCores, NumberOfLogicalProcessors
- wmic cpu get name, NumberOfCores, NumberOfLogicalProcessors
- Get-PSDrive -PSProvider FileSystem

Below query gives .net version if Output is 0x82348 (**533320** (in decimal)) then convert to decimal and check if it is the required version

reg query "HKLM\SOFTWARE\Microsoft\NET Framework Setup\NDP\v4\Full" /v Release

```
$Release = (Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\NET Framework
Setup\NDP\v4\Full").Release;
if ($Release -ge 533320) {"4.8.1"}
elseif ($Release -ge 528040) {"4.8"}
elseif ($Release -ge 461808) {"4.7.2"}
elseif ($Release -ge 461308) {"4.7.1"}
elseif ($Release -ge 460798) {"4.7"}
elseif ($Release -ge 394802) {"4.6.2"}
elseif ($Release -ge 394254) {"4.6.1"}
elseif ($Release -ge 393295) {"4.6"}
elseif ($Release -ge 379893) {"4.5.2"}
elseif ($Release -ge 378675) {"4.5.1"}
elseif ($Release -ge 378389) {"4.5"}
else {"Lower than 4.5 or not installed"}
```

\$PSVersionTable.PSVersion

- SELECT SERVERPROPERTY('Edition'), SERVERPROPERTY('ProductVersion');
- fsutil fsinfo volumeinfo C: -- Confirm file system is **NTFS** (SQL Server doesn't support FAT32/ReFS for data files).

2. Verify Windows OS is supported.

- systeminfo | findstr /B /C:"OS Name" /C:"OS Version" -- Check Your Current OS Version Compare with Supported OS List for SQL Server
- <https://learn.microsoft.com/en-us/sql/sql-server/install/hardware-and-software-requirements-for-installing-sql-server?view=sql-server-ver17>
- Check OS Service Pack or Build Level Some older OS versions (like Windows Server 2012 R2) require specific service packs.
systeminfo | find "Service Pack"

3. Verify SQL Server edition and license compatibility.

- Get-ItemProperty -Path 'HKLM:\SOFTWARE\Microsoft\Microsoft SQL Server\MSSQL*\Setup' | Select-Object InstanceName, Edition, Version, SQLBinRoot
- **SELECT** SERVERPROPERTY('MachineName') **AS** [MachineName],

```

SERVERPROPERTY('ServerName') AS [ServerName],
SERVERPROPERTY('Edition') AS [Edition],
SERVERPROPERTY('ProductVersion') AS [ProductVersion],
SERVERPROPERTY('ProductLevel') AS [ServicePackLevel],
SERVERPROPERTY('EngineEdition') AS [EngineEdition],
SERVERPROPERTY('LicenseType') AS [LicenseType],
SERVERPROPERTY('NumLicenses') AS [NumLicenses];

```

4. Verify SQL Server build supports upgrade path.

Check for Minimum Required Service Pack (SP) Level

Source Version	Minimum SP Required for Upgrade
SQL Server 2014	SP2 or later
SQL Server 2016	SP1 or later
SQL Server 2017	RTM or later (no SP required)
SQL Server 2019	RTM or later (for 2022 upgrade)

Always install the **latest Cumulative Update (CU)** on the source before upgrading — this ensures smoother metadata conversions and fewer setup errors.

<https://learn.microsoft.com/en-us/sql/database-engine/install-windows/supported-version-and-edition-upgrades-2022?view=sql-server-ver17>

5. Verify SQL Server instance name and features to upgrade.

- Get-Service | Where-Object {\$_.Name -like "MSSQL*"} | Select-Object Name, DisplayName, Status
- Get-WmiObject -namespace "root\Microsoft\SqlServer\ComputerManagement14" -class SqlServiceAdvancedProperty | Where-Object { \$_.PropertyName -eq "FEATURE" } | Select-Object SqlServiceType, ServiceName, PropertyStringValue -- need PS

6. Verify sufficient disk space on all drives (data, log, temp, backup).

```

SELECT
    DB_NAME(database_id) AS [DatabaseName],
    type_desc AS [FileType],
    physical_name AS [FilePath]
FROM sys.master_files
ORDER BY DB_NAME(database_id), type_desc;

```

Note which drives store: Data files (MDF/NDF), Log files (LDF), TempDB files, Backup location

- Check free space using WMI

```

Get-WmiObject Win32_LogicalDisk -Filter "DriveType=3" | Select-Object DeviceID,
VolumeName, @{Name="FreeGB";Expression={([math]::Round($_.FreeSpace/1GB,2))}},
@{Name="TotalGB";Expression={([math]::Round($_.Size/1GB,2))}}

```

- This helps estimate growth during upgrade and ensure sufficient space.

```

EXEC sp_MSforeachdb '
USE [?];
SELECT DB_NAME() AS DatabaseName,
SUM(size*8/1024) AS SizeMB

```

```
FROM sys.master_files GROUP BY type_desc;
';
```

7. Verify backups folder has free space.
8. Run **SELECT @@VERSION** and note current version.
9. Record SQL instance collation.

```
SELECT
    SERVERPROPERTY('Collation') AS [InstanceCollation],
    SERVERPROPERTY('InstanceName') AS [InstanceName],
    SERVERPROPERTY('ProductVersion') AS [ProductVersion],
    SERVERPROPERTY('Edition') AS [Edition],
    SERVERPROPERTY('ServerName') AS [ServerName];
```

To ensure no DB has a different collation

```
SELECT
    name AS [DatabaseName],
    collation_name AS [DatabaseCollation]
FROM sys.databases
    ORDER BY name;
```

10. Record SQL authentication mode.

```
DECLARE @AuthMode INT;
EXEC master.dbo.xp_instance_regread
    N'HKEY_LOCAL_MACHINE',
    N'SOFTWARE\Microsoft\MSSQLServer\MSSQLServer',
    N'LoginMode',
    @AuthMode OUTPUT;

SELECT CASE @AuthMode
    WHEN 1 THEN 'Windows Authentication'
    WHEN 2 THEN 'Windows and SQL Server Authentication (Mixed Mode)'
    ELSE 'Unknown'
END AS [AuthenticationMode];
```

If your instance uses **Mixed Mode**, ensure the sa login is enabled before upgrade:

```
SELECT name, is_disabled FROM sys.sql_logins WHERE name = 'sa';
```

You can also record all SQL-authenticated logins (helps during post-upgrade validation):

```
SELECT name, type_desc, create_date, is_disabled
FROM sys.server_principals
WHERE type_desc = 'SQL_LOGIN';
```

11. Record server name and IP.

- Hostname ---server name
- Ipconfig -- Server IP Address
- Using SQL Server Query:

```
SELECT SERVERPROPERTY('MachineName') AS [ServerName],

        SERVERPROPERTY('ServerName') AS [SQLInstanceName];
```

12. Record SQL Server ports.

```
DECLARE @port NVARCHAR(10);
EXEC xp_instance_regread
    'HKEY_LOCAL_MACHINE',
```

```
'Software\Microsoft\Microsoft SQL
Server\MSSQLServer\SuperSocketNetLib\Tcp\IPAll',
'TcpPort',
@port OUTPUT;
SELECT 'SQL Server Port' AS [Description], @port AS [Port];
```

Run the following T-SQL query in each instance. This works for **default and named instances** — it reads the registry key for the instance in which the query is run.

13. Record SQL Server service accounts and passwords.

PowerShell command on the SQL Server:

```
Get-WmiObject win32_service | Where-Object {$_.Name -like "MSSQL*" -or $_.Name -like
"SQLAgent*" -or $_.Name -like "SQLBrowser"} | Select-Object Name, StartName, State,
StartMode
```

Using SQL Query (For Limited Info):

```
SELECT servicename, service_account, status_desc
FROM sys.dm_server_services;
```

14. Record SQL Agent jobs.

Script Out All SQL Agent Jobs (Recommended)

1. Open Object Explorer → SQL Server Agent → Jobs.
2. Right-click each job → Script Job as → CREATE To → File.
3. Save all .sql scripts to a backup folder

15. Record linked servers.

```
SELECT * FROM sys.servers
```

16. Record logins and permissions.

17. Record server-level configurations (sp_configure).

```
SELECT
'EXEC sp_configure '' + name + ''', ' + CAST(value AS VARCHAR(20)) + ' ';
RECONFIGURE WITH OVERRIDE;' AS [RecreateScript]
FROM sys.configurations
ORDER BY name;
```

Before running this, ensure advanced options are visible. Then rerun your generation script — this ensures you capture everything

```
SELECT name, value, value_in_use
FROM sys.configurations
WHERE name = 'show advanced options';

SELECT * FROM sys.configurations;
```

18. Record SQL Server max memory setting.

```
SELECT
name AS [ConfigurationName],
value_in_use AS [CurrentValue_MB]
```

```
FROM sys.configurations
WHERE name IN ('max server memory (MB)', 'min server memory (MB)');
```

19. Record tempdb configuration.

Query TempDB File Configuration

```
USE tempdb;
GO
SELECT
    DB_NAME(database_id) AS [DatabaseName],
    name AS [LogicalName],
    type_desc AS [FileType],
    physical_name AS [FilePath],
    size * 8 / 1024 AS [Size_MB],
    growth AS [GrowthValue],
    CASE
        WHEN is_percent_growth = 1 THEN 'Percent (' + CAST(growth AS VARCHAR(10)) +
        '%)'
        ELSE 'MB (' + CAST(growth * 8 / 1024 AS VARCHAR(10)) + ' MB)'
    END AS [AutoGrowth],
    state_desc AS [Status],
    max_size,
    file_id
FROM sys.master_files
WHERE database_id = DB_ID('tempdb')
ORDER BY type_desc, file_id;
```

Generate Script to Recreate TempDB Configuration

```
USE tempdb;
GO
SELECT
    'ALTER DATABASE tempdb MODIFY FILE (NAME = [' + name +
    '], FILENAME = N''' + physical_name +
    ''', SIZE = ' + CAST(size * 8 / 1024 AS VARCHAR(10)) + 'MB, FILEGROWTH = ' +
    CASE
        WHEN is_percent_growth = 1 THEN CAST(growth AS VARCHAR(10)) + '%'
        ELSE CAST(growth * 8 / 1024 AS VARCHAR(10)) + 'MB'
    END + ');' AS [RecreateScript]
FROM sys.master_files
WHERE database_id = DB_ID('tempdb')
ORDER BY file_id;
```

20. Record database recovery models, database compatibility levels

```
SELECT
    name AS [DatabaseName],
    recovery_model_desc AS [RecoveryModel],
    compatibility_level AS [CompatibilityLevel]
FROM sys.databases
ORDER BY name;
```

21. Record replication, mirroring, AG, log shipping configurations.

REPLICATION CHECK:

```
SELECT is_published, is_subscribed, is_distributor
FROM sys.databases
WHERE name NOT IN ('master', 'model', 'msdb', 'tempdb');
```

- is_published = 1 → This DB is a Publisher
- is_subscribed = 1 → This DB is a Subscriber
- is_distributor = 1 → This DB is a Distributor

You can use **SQL Server Management Studio (SSMS)** →
Right-click **Replication** → **Generate Scripts** →
Choose **All Publications and Subscriptions** to export as .sql file.

Query to Check Mirroring

```
SELECT
    DB_NAME(database_id) AS [DatabaseName],
    mirroring_state_desc,
    mirroring_partner_name AS [PartnerServer],
    mirroring_role_desc,
    mirroring_witness_name AS [WitnessServer]
FROM sys.database_mirroring
WHERE mirroring_guid IS NOT NULL;
```

AG:

AG and Replica Information

```
SELECT
    ag.name AS [AGName],
    agd.database_name AS [DatabaseName],
    ar.replica_server_name AS [ReplicaServer],
    ars.role_desc AS [ReplicaRole],
    agl.dns_name AS [ListenerName],
    agl.port AS [ListenerPort]
FROM sys.availability_groups ag
JOIN sys.availability_databases_cluster agd ON ag.group_id = agd.group_id
JOIN sys.availability_replicas ar ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states ars ON ar.replica_id =
ars.replica_id
LEFT JOIN sys.availability_group_listeners agl ON ag.group_id = agl.group_id
ORDER BY ag.name, agd.database_name;
```

Replica Endpoints

```
SELECT
    ar.replica_server_name,
    ep.name AS [EndpointName],
    ep.state_desc AS [EndpointState],
    ep.type_desc AS [EndpointType],
    ep.protocol_desc,
    ep.port,
    ep.role_desc
FROM sys.availability_replicas ar
JOIN sys.database_mirroring_endpoints ep
ON ar.endpoint_url LIKE '%' + CAST(ep.port AS VARCHAR(5)) + '%';
```

LOGG SHIPPING

```
USE msdb;
GO
SELECT
    primary_server,
    primary_database,
    backup_directory,
    backup_share,
    secondary_server,
    secondary_database,
    restore_mode,
    monitor_server,
    last_backup_file,
    last_copied_file,
    last_restored_file
FROM dbo.log_shipping_primary_databases AS prim
JOIN dbo.log_shipping_primary_secondaries AS sec
ON prim.primary_id = sec.primary_id;
```


22. Record Maintenance plans.
23. Record SSIS packages, if used.
24. Record SSRS configuration and encryption key.
25. Record Full-Text catalogs.

Check if Full-Text Search Feature Is Installed

```
SELECT FULLTEXTSERVICEPROPERTY('IsFullTextInstalled') AS IsFullTextInstalled;
SELECT * FROM sys.fulltext_catalogs
```

--List Full-Text Indexes on Tables

```
SELECT
    OBJECT_SCHEMA_NAME(i.object_id) AS [SchemaName],
    OBJECT_NAME(i.object_id) AS [TableName],
    c.name AS [ColumnName],
    i.name AS [FullTextIndexName],
    ic.type_description AS [ColumnType],
    ftc.name AS [FullTextCatalog],
    fti.change_tracking_state_desc AS [ChangeTracking]
FROM sys.fulltext_indexes fti
JOIN sys.fulltext_catalogs ftc ON fti.fulltext_catalog_id = ftc.fulltext_catalog_id
JOIN sys.fulltext_index_columns fic ON fti.object_id = fic.object_id
JOIN sys.columns c ON fic.column_id = c.column_id AND fic.object_id = c.object_id
JOIN sys.indexes i ON fti.unique_index_id = i.index_id AND fti.object_id = i.object_id
JOIN sys.types ic ON c.user_type_id = ic.user_type_id
ORDER BY [SchemaName], [TableName];
```

-- Script to Recreate Full-Text Catalogs

```
SELECT
    'CREATE FULLTEXT CATALOG [' + name + ']'
    + CASE WHEN is_default = 1 THEN ' AS DEFAULT' ELSE '' END
    + ' AUTHORIZATION [dbo];' AS [CreateScript]
FROM sys.fulltext_catalogs;
```

-- Script to Recreate Full-Text Indexes

```
SELECT
    'CREATE FULLTEXT INDEX ON [' + OBJECT_SCHEMA_NAME(fti.object_id) + '].[' +
    OBJECT_NAME(fti.object_id) + '] (' +
    STRING_AGG([' + c.name + '] LANGUAGE 1033', ', ') + ') ' +
    ' KEY INDEX [' + i.name + '] ON [' + ftc.name + '];' AS [CreateScript]
FROM sys.fulltext_indexes fti
JOIN sys.fulltext_catalogs ftc ON fti.fulltext_catalog_id = ftc.fulltext_catalog_id
JOIN sys.fulltext_index_columns fic ON fti.object_id = fic.object_id
JOIN sys.columns c ON fic.column_id = c.column_id AND fic.object_id = c.object_id
JOIN sys.indexes i ON fti.unique_index_id = i.index_id AND fti.object_id = i.object_id
GROUP BY fti.object_id, i.name, ftc.name;
```

26. Record Database Mail profiles.

```
EXEC sp_configure 'Database Mail XPs';
```

```
SELECT *
FROM msdb.dbo.sysmail_profile;
```

```
SELECT
    p.name AS [ProfileName],
    a.name AS [AccountName],
    pa.sequence_number AS [SequenceNumber]
FROM msdb.dbo.sysmail_profileaccount pa
JOIN msdb.dbo.sysmail_profile p ON pa.profile_id = p.profile_id
JOIN msdb.dbo.sysmail_account a ON pa.account_id = a.account_id
```

```
ORDER BY p.name, pa.sequence_number;
```

```
SELECT *
FROM msdb.dbo.sysmail_allitems
ORDER BY sent_date DESC;
```

Use can also finds this in the database mail section screenshot those

27. Record startup parameters.

SQL Server Management Studio

1. Open **SSMS** → Connect to the SQL instance.
2. Right-click **Server Name** → **Properties** → **Startup Parameters**.
3. Record all values listed there

```
SELECT *
FROM sys.dm_server_registry
```

Why It's Important

1. **Master Database Path (-d)** — Incorrect path → SQL Server won't start.
 2. **Error Log Path (-e)** — For troubleshooting post-upgrade.
 3. **Trace Flags (-T)** — Some flags are deprecated in newer versions.
 4. **TempDB Path** — Needed for performance and multi-file configurations.
 5. **Single-user or minimal config flags (-m/-f)** — Ensure they are not accidentally enabled.
- Always **validate these parameters post-upgrade** to ensure SQL Server starts properly.

28. Record existing cluster configuration (if applicable).

29. Record firewall rules for SQL ports.

30. Backup all databases (Full + Log + System DBs).
31. Backup SQL Agent jobs (via script).
32. Backup logins (via script).

```
SELECT 'EXEC sp_addsrvrolemember ''' + l.name + ''', ''' + r.name + ''';' + CHAR(13) +
'GO'
FROM sys.server_role_members m
JOIN sys.server_principals r ON m.role_principal_id = r.principal_id
JOIN sys.server_principals l ON m.member_principal_id = l.principal_id
ORDER BY l.name, r.name;
```

```
-- =====
-- Script: Backup all SQL Server logins + roles
-- Purpose: Generate a SQL script to recreate logins
-- Author: Pre-upgrade audit
-- =====
```

```
-- [1] Backup SQL Logins (SQL Server logins)
```

```
SELECT
    'CREATE LOGIN [' + name + '] WITH PASSWORD = 0x' +
    CONVERT(VARCHAR(MAX), password_hash, 1) +
    ' HASHED, CHECK_POLICY = ' + CASE WHEN is_policy_checked = 1 THEN 'ON' ELSE
    'OFF' END +
    ', CHECK_EXPIRATION = ' + CASE WHEN is_expiration_checked = 1 THEN 'ON' ELSE
    'OFF' END + ';' + CHAR(13) + 'GO'
FROM sys.sql_logins
WHERE name NOT LIKE '##%' -- exclude system logins
ORDER BY name;
```

```
-- [2] Backup Windows Logins
```

```
SELECT
    'CREATE LOGIN [' + name + '] FROM WINDOWS;' + CHAR(13) + 'GO'
FROM sys.server_principals
WHERE type IN ('U', 'G') -- Windows user or group
```

```

        AND name NOT LIKE 'BUILTIN\%'
ORDER BY name;

-- [3] Backup Server Role Memberships
SELECT
    'EXEC sp_addsrvrolemember ''' + l.name + ''', ''' + r.name + ''';' + CHAR(13)
    + 'GO'
FROM sys.server_role_members m
JOIN sys.server_principals r ON m.role_principal_id = r.principal_id
JOIN sys.server_principals l ON m.member_principal_id = l.principal_id
ORDER BY l.name, r.name;

```

33.Backup linked servers (via script).

--Check Existing Linked Servers

```
EXEC sp_linkedservers;
```

--Script Linked Servers (T-SQL) The following script generates a recreation script for all linked servers:

```

SELECT
    'EXEC sp_addlinkedserver
        @server = N''' + name + ''',
        @srvproduct = N''' + ISNULL(srvproduct, '') + ''',
        @provider = N''' + provider + ''',
        @datasrc = N''' + ISNULL(datasrc, '') + ''';' + CHAR(13) + 'GO'
FROM sys.servers
WHERE is_linked = 1
ORDER BY name;

```

-- Script Linked Server Security Mappings. Most linked servers also have login mappings:

```

SELECT
    'EXEC sp_addlinkedsrvlogin
        @rmtsrvname = N''' + s.name + ''',
        @useself = ' + CASE WHEN l.useself = 1 THEN 'N''True''' ELSE 'N''False''' END +
        ',
        @locallogin = N''' + ISNULL(l.local_principal_name, '') + ''',
        @rmtuser = N''' + ISNULL(l.remote_principal_name, '') + ''',
        @rmtpassword = N''' + ISNULL(l.remote_password, '') + ''';' + CHAR(13) + 'GO'
FROM sys.servers s
JOIN sys.linked_logins l ON s.server_id = l.server_id
WHERE s.is_linked = 1
ORDER BY s.name;

```

34.Backup SSISDB and packages.

1. Backup SSISDB and encryption key before upgrading SQL Server.
2. Keep encryption key and password safe; without it, sensitive data in packages is unrecoverable.
3. For multi-server environments, backup SSISDB on the primary server.
4. Optionally export projects/packages individually as .ispac files.
5. Test restore on a non-production server to ensure everything works.

```

USE SSISDB;
GO

```

```

-- Backup encryption key
DECLARE @BackupFile NVARCHAR(4000) = N'C:\Backup\SSISDB_KEY_BACKUP.key';

```

```

DECLARE @Password NVARCHAR(128) = N'MyStrongPassword123!'; -- choose a secure
password

EXEC [catalog].[backup_encryption_key]
    @password = @Password,
    @encryption_key_path = @BackupFile;
GO

```

35. Backup SSRS encryption key.

- Open SQL Server Reporting Services Configuration Manager.
- Connect to the report server instance.
- Go to Encryption Keys → Click Backup.
- Choose a secure location to save the key file (e.g., C:\Backup\SSRS_EncryptionKey.snk).
- Enter a strong password to encrypt the backup.
- Click OK and verify backup completed successfully.

Restore the Key After Upgrade

1. Open **SSRS Configuration Manager** → **Encryption Keys** → **Restore**.
2. Provide the **backup file** and **password**.
3. Click **OK** to restore.

36. Backup master, msdb, model databases.

37. Backup all certificates and keys.

- Database encryption (TDE)
- Service Broker
- SSL/TLS connections
- Backup encryption
- SSRS/SSIS sensitive data

If these certificates/keys are not backed up, encrypted data may become inaccessible after the upgrade.

- Backup SMK first, then DMKs, then certificates/keys.
- Use strong passwords for private keys.
- Store backups in a secure location, preferably off-server.
- Test restore on a non-production instance before upgrade.
- For SSRS, SSIS, and linked servers with encryption, ensure you also backup service keys.

--Backup Database Encryption Certificates (TDE)

--Identify encrypted databases:

```

SELECT name, is_encrypted
FROM sys.databases
WHERE is_encrypted = 1;

```

--Backup the certificate used for TDE:

```

USE master;
GO
-- Replace with your certificate name
BACKUP CERTIFICATE [YourCertificateName]
TO FILE = 'C:\Backup\YourCertificateName.cer'
WITH PRIVATE KEY (
    FILE = 'C:\Backup\YourCertificateName_PrivateKey.pvk',
    ENCRYPTION BY PASSWORD = 'StrongPasswordHere'
);

```

GO

--Backup Service Master Key. The Service Master Key (SMK) is the root of SQL Server encryption hierarchy:
--Needed to restore all dependent keys.
--Store key file and password securely.

-- Backup Service Master Key

USE master;

GO

BACKUP SERVICE MASTER KEY TO FILE = 'C:\Backup\SMK_Backup.key'

 ENCPTION BY PASSWORD = 'StrongPasswordHere';

GO

--Backup Database Master Keys (DMK)

--Each database with encrypted objects (TDE, symmetric keys, credentials) has a DMK:
--Each DMK backup is per database, password-protected.
--Required to restore TDE keys, symmetric keys, and credentials.

-- Loop through all databases with DMK

DECLARE @DBName NVARCHAR(128);

DECLARE db_cursor CURSOR FOR

SELECT name

FROM sys.databases

WHERE state_desc = 'ONLINE';

OPEN db_cursor;

FETCH NEXT FROM db_cursor INTO @DBName;

WHILE @@FETCH_STATUS = 0

BEGIN

 DECLARE @SQL NVARCHAR(MAX);

 SET @SQL = 'USE [' + @DBName + ']; ' +

 'IF EXISTS (SELECT * FROM sys.symmetric_keys WHERE name =

 '##MS_DatabaseMasterKey##') ' +

 'BEGIN ' +

 'BACKUP MASTER KEY TO FILE = 'C:\Backup\DMK_' + @DBName + '.key' ' +

 +

 'ENCPTION BY PASSWORD = 'StrongPasswordHere'; ' +

 'END';

 EXEC (@SQL);

 FETCH NEXT FROM db_cursor INTO @DBName;

END

CLOSE db_cursor;

DEALLOCATE db_cursor;

--Backup Certificates and Asymmetric Keys in Databases

--Prints backup commands for each certificate.

--Execute the printed commands to create .cer and .pvk files.

-- Backup all certificates with private keys

USE [YourDatabase];

GO

DECLARE @certName NVARCHAR(128);

DECLARE cert_cursor CURSOR FOR

SELECT name FROM sys.certificates;

OPEN cert_cursor;

FETCH NEXT FROM cert_cursor INTO @certName;

WHILE @@FETCH_STATUS = 0

BEGIN

 DECLARE @SQL NVARCHAR(MAX);

 SET @SQL = 'BACKUP CERTIFICATE [' + @certName + ']' +

 'TO FILE = 'C:\Backup\Cert_' + @certName + '.cer' ' +

 'WITH PRIVATE KEY (FILE = 'C:\Backup\Cert_' + @certName +

 '_PrivateKey.pvk', ENCRYPTION BY PASSWORD = 'StrongPasswordHere');';

```

PRINT @SQL;
FETCH NEXT FROM cert_cursor INTO @certName;
END

CLOSE cert_cursor;
DEALLOCATE cert_cursor;

```

38. Backup SQL registry keys (HKLM\Software\Microsoft\Microsoft SQL Server).

Backup Registry Keys Using Regedit

1. Press Win + R, type regedit, press Enter.
2. Navigate to:

HKKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Microsoft SQL Server

3. Right-click → **Export**.
4. Save file as C:\Backup\SQLServerRegistry_Backup.reg.
5. Choose **All** under export range to capture the complete key.

Keep the .reg file in a **secure location** (off-server if possible).

Restore Registry Keys Using Regedit

⚠ Make sure SQL Server services are **stopped** before doing this.

1. Open **Registry Editor** (Win + R, type regedit, Enter).
2. Click **File** → **Import...**
3. Select your **backup file** (SQLServerRegistry_Backup.reg).
4. Click **Open**.
5. The registry will merge keys from the backup.
6. SQL Server services start

39. Backup SQL configuration files.
40. Run Microsoft Data Migration Assistant (DMA).
41. Resolve all blocking/deprecated issues found by DMA.
42. Disable replication, log shipping, and Always On (if configured).
43. Stop all ETL jobs, third-party monitoring tools, and backups.
44. Notify application owners and stakeholders.
45. Stop user activity or switch to maintenance mode.
46. Take a system snapshot or VM backup.

⚙ 2. UPGRADE EXECUTION

51. Log in as local administrator.
52. Disable antivirus temporarily.
53. Mount SQL Server installation media.
54. Run setup.exe.
55. Choose **Upgrade from a previous version of SQL Server**.
56. Setup Support Rules → Verify all passed.
57. Select instance to upgrade.
58. Select features to upgrade (Database Engine, SSAS, SSRS, etc.).
59. Check for feature rules validation.
60. Accept license terms.
61. Choose service accounts (keep existing).
62. Choose authentication mode (keep existing).
63. Verify upgrade rules → all passed.

64. Begin upgrade.
 65. Monitor progress until completion.
 66. Verify successful upgrade message.
 67. Close setup.
-

3. POST-UPGRADE STEPS

68. Restart SQL Server services.
69. Verify SQL Server version:
70. `SELECT @@VERSION;`
71. Check instance name and edition:
72. `SELECT SERVERPROPERTY('ProductVersion'), SERVERPROPERTY('Edition');`
73. Open SQL Server Configuration Manager → confirm all services started.
74. Verify SQL Server Agent service.
75. Verify Database Engine service.
76. Verify SQL Browser service.
77. Check Event Viewer → SQL logs → setup summary.

Run:

`DBCC CHECKDB;`

Run:

`EXEC sp_configure;`

78. Verify max server memory and reapply if reset.
79. Reapply any custom configuration values.
80. Verify all databases are online.
81. Verify SQL Agent jobs exist and enabled.
82. Verify logins and permissions restored.
83. Verify linked servers working.
84. Verify mail profiles working.
85. Verify SSIS packages working.
86. Restore SSRS encryption key (if applicable).
87. Verify Reporting Services URL access.
88. Verify replication configuration restored.
89. Re-enable Always On or log shipping.
90. Recreate database mirroring if used.
91. Recreate maintenance plans.
92. Test backups.
93. Test index rebuild and DBCC jobs.
94. Validate alerts and operators.
95. Validate performance baseline.
96. Test application connectivity.
97. Validate connection strings.
98. Validate authentication (SQL + Windows).
99. Validate collation consistency.

Verify tempdb file configuration.

Update statistics:

`sql EXEC sp_updatestats;`

Check compatibility levels:

`sql SELECT name, compatibility_level FROM sys.databases;`

100. Keep at old level temporarily; raise later after testing.
101. Reconfigure server memory and MAXDOP settings.

102. Rebuild indexes.
 103. Run query performance comparison (old vs new).
 104. Review Error Logs for upgrade messages.
 105. Review C:\Program Files\Microsoft SQL Server\<version>\Setup Bootstrap\Log\Summary.txt.
 106. Re-enable antivirus.
 107. Enable monitoring tools and jobs.
 108. Take a fresh full backup of all databases.
 109. Document new build number.
 110. Verify SQL Agent history clean.
 111. Run full integrity test.
 112. Validate database mail test message.
 113. Verify cluster resources online (if applicable).
 114. Failover test (if Always On).
 115. Validate application end-to-end testing (UAT).
 116. Inform stakeholders of successful upgrade.
 117. Archive all pre-upgrade and post-upgrade logs to C:\Upgrade.
-

4. POST-UPGRADE OPTIMIZATION (OPTIONAL)

121. Change compatibility levels to latest version.
 122. Run performance baseline checks.
 123. Enable Query Store.
 124. Analyze Query Store regressions.
 125. Update database statistics again.
 126. Run index optimize job.
 127. Apply latest SQL cumulative update (CU).
 128. Verify build after CU.
 129. Run DBCC CHECKDB again after CU.
 130. Re-take full backup.
 131. Update documentation and DR runbooks.
 132. Enable automated maintenance plans.
-